SURGET.AI ENGINEERING & RESEARCH HANDBOOK

Surget.AI: Beginner's Complete Tech & CUA Guide

Python, NumPy, Vectors, Embeddings, FAISS, Agent Architecture, GUI Grounding & Research Roadmap

Purpose of this Document: Agar aap AI agents aur machine learning mein naye hain, toh yeh document aapko batata hai ki is project mein *Python se lekar FAISS aur Computer-Use Agents (CUA) tak* har ek cheez ka asal mein kya role hai, kya-kya padhna hai, aur practical experiments kaise run karne hain.

1. Foundation Tools & Concepts: Kya Cheez Kahan Use Hogi?

1. Python — Central Nervous System

Is Project Mein Kaam: Poore agent ka master glue. OS se buttons read karna (`pywinauto`), mouse/keyboard actions lena (`pyautogui`), models run karna aur workflow execution loop manage karna.

2. NumPy — Pixel & Matrix Engine

Is Project Mein Kaam: 1080p screen asal mein 1920×1080×3 ka numbers matrix hoti hai. NumPy se frame differencing, coordinates transform, aur vector math lightning speed (C-speed) par hoti hai.

3. Vector Operations — Spatial & Similarity Math

Is Project Mein Kaam: Bounding boxes ka center nikalna `[(x1+x2)/2, (y1+y2)/2]`, frames subtract karke badle hue pixels detect karna, aur dot products se task similarity nikalna.

4. Probability & Classification — Decision Confidence

Is Project Mein Kaam: Screen ke boxes classify karna (Button vs Textfield). Model ka confidence score (≥0.85) check karke decide karna: fast click karein ya dobara inspect karein.

5. Embeddings (MiniLM) — Meaning as Numbers

Is Project Mein Kaam: Text descriptions ko 384-dimensional vector mein badalna. Jaise "*Submit form*" aur "*Send completed form*" ke vectors geometrically ek doosre ke bohot paas hote hain.

6. Cosine Similarity — Directional Matcher

Is Project Mein Kaam: Formula: `dot(A, B) / (norm(A) * norm(B))`. User ki query vector ko memory ke past workflow vectors se match karke sabse close match (+1.0) retrieve karna.

7. PyTorch — Deep Learning Runtime

Is Project Mein Kaam: Tensors aur GPU acceleration. Local neural models jaise YOLOv8 (icon detector), PaddleOCR, aur local small VLMs ko CPU/GPU par fast inference run karna.

8. FAISS (Facebook AI) — Vector Database Index

Is Project Mein Kaam: Jab company ke 10,000+ workflows save ho jate hain, tab NumPy slow ho jata hai. FAISS `IndexFlatIP` microsecond (<0.05ms) mein nearest workflow dhoondh nikaalta hai.

2. Real Semantic Search & Workflow Memory Retrieval

AI agent har baar scratch se nahi sochta. Pehle `all-MiniLM-L6-v2` embedding model se check karta hai ki kya usne yeh task pehle kiya hai:

```
from sentence_transformers import SentenceTransformer
import numpy as np

# 1. Real pretrained lightweight embedding model load karo (Runs on CPU in 5ms)
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

# 2. Agent ka Workflow Memory Database
memory = [
    {"id": "wf_001", "task": "Click Submit to send completed customer form", "action": "click_submit"},
    {"id": "wf_002", "task": "Save current spreadsheet document before closing", "action": "click_save"},
    {"id": "wf_003", "task": "Search for a customer account in CRM", "action": "search_customer"},
]

# 3. Store normalized vectors
vectors = model.encode([m["task"] for m in memory], normalize_embeddings=True)

# 4. User Query: "I want to send my completed invoice form"
query = "I want to send my completed invoice form"
q_vec = model.encode([query], normalize_embeddings=True)[0]

# 5. Cosine similarity via dot product
scores = vectors @ q_vec
best_idx = np.argmax(scores)

print(f"Matched: {memory[best_idx]['id']} | Score: {scores[best_idx]:.4f} | Action: {memory[best_idx]
['action']}")
# Output: Matched: wf_001 | Score: 0.8841 | Action: click_submit
```

PRACTICAL RESEARCH NUANCE

Sirf text match hona kaafi nahi hota! Real agent mein hum **Preconditions** bhi check karte hain: Kya sahi software khula hai? Kya target button screen par actually visible hai? Iske baad hi fast action execute hota hai.

3. Practical ML Experiments: Evaluation & Metrics

Research ka matlab hai data ke sath prove karna ki hamara method baseline se better hai:

Experiment A: Retrieval Quality

Search Precision

Precision@k: Top-k retrieved results mein se kitne relevant hain?
Recall@k: Total relevant workflows mein se kitne top-k mein mile?
MRR (Mean Reciprocal Rank): Sahi workflow pehli rank par aaya ya doosri par? (Target: > 0.85).

Experiment B: Agent Speed & Efficiency

Latency & Cost

Task Success Rate (%): Poora task bina error ke khatam hua?
Median Latency (ms): Typical action kitna fast chala (8ms vs 3500ms)?
p95 Latency (ms): Worst 5% cases kitne slow the?
Model Invocations: Ek task mein kitni baar expensive VLM call hua?

4. Computer-Use Agent (CUA) Architecture & Execution Loop

A. Observe → Plan → Act → Verify (The 4-Step Loop)

1. **OBSERVE:** Screen capture karo (`mss`), OS window check karo, DOM/UIA tree inspect karo.
2. **PLAN:** Next sub-goal decide karo (e.g., "*Search box dhoondho aur query type karo*").
3. **ACT:** Hardware primitive dispatch karo (Mouse click, type text, shortcut key).
4. **VERIFY: (Sabse Zaroori)** Click ke baad expected result aaya ya nahi? Agar nahi aaya, recover karo!

B. Tool Calling & State Management

- **Tool Calling:** LLM direct mouse nahi chalata. Woh structured JSON emit karta hai: `{"action": "click", "target": "btn_submit", "coords": [450, 120]}`. Python code use validate karke execute karta hai.
- **State Management:** Agent ko pata hota hai ki current app kaunsa hai, previous action kya tha, retry count kitna hua, aur task status kya hai.

C. Tri-Tier GUI Grounding (Screen Par Button Dhoondhna)

Grounding Tier	Kahan Se Data Milta Hai?	Speed (Latency)	Cost & Accuracy
Tier-0: OS Accessibility	Windows UIA / Chrome DOM	**< 10 ms**	$0.00 | 100% Deterministic (Zero Hallucination)
Tier-1: Fast Local Vision	OpenCV Dirty Region + PaddleOCR	**50 – 80 ms**	$0.00 | 95%–98% (Canvas/Photoshop apps)
Tier-2: Foundation VLM	UI-TARS / Claude 3.5 / GPT-4o	**2,000 – 4,000 ms**	Active Token API | High Reasoning Fallback

D. Failure Recovery & Confidence-Based Routing

- Agar Tier-0 match confidence ≥ 0.90 → Direct Instant Click!
- Agar button move ho gaya ya pop-up aa gaya → **Sentinel Assertion** pakad legi.
- Agent blindly repeat nahi karega. Fresh observation lega, Tier-2 VLM se self-healing patch karega, aur memory update kar lega.

5. 5 Core Research Papers (Isi Order Mein Padhna Hai)

Paper	Authors / Lab	Is Paper Ka Core Purpose	Surget.AI Mein Iska Role
1. OSWorld	CMU / HKU (NeurIPS'24)	Real OS environments (Ubuntu/Windows) par tasks ka standard benchmark.	Hamara primary testing benchmark jahan hum SOTA prove karenge.
2. SeeClick	Nanjing / Shanghai AI Lab	Screenshot se direct normalized (x, y) coordinates regression.	ScreenSpot benchmark use karke visual grounding accuracy test karenge.
3. OmniParser	Microsoft Research (2024)	YOLOv8 + Florence-2 se buttons par Set-of-Marks number tags lagana.	Iska icon bounding concept Tier-2 fallback mein integrate kiya gaya.
4. Agent S2	Simular AI (2025)	Mixture of Grounding (MoG) voting ensemble + Manager/Worker hierarchy.	Iske perception ensemble ko humne Tri-Tier + Workflow Memory mein upgrade kiya.
5. UI-TARS	ByteDance Seed (2025)	Native End-to-End GUI VLM with System-2 Thinking tokens.	Hamara Tier-2 Emergency Specialist jab koi complex visual reasoning chahiye ho.

6. Practical 30-Day Step-by-Step Learning & Action Plan

Days 1 – 7: Python & Native OS Hooks

Foundation

- Python classes, dictionaries, async.
- `pywinauto` se Windows ke button dhoondhna.
- `mss` se 5ms mein screenshot capture.
- *Task:* Notepad ka button bina screenshot ke click karna.

Days 8 – 14: Embeddings & FAISS

Memory

- `sentence-transformers` se vectors banana.
- Cosine similarity se workflow retrieval.
- FAISS IndexFlatIP setup.
- *Task:* 10 test queries par Precision@k aur Recall@k nikalna.

Days 15 – 21: Browser CUA Testbed

Baseline

- Playwright se 10 repeatable tasks banana (Login, Search, Tables).
- Baseline agent run karna (Screenshot → Model → Click).
- *Task:* Baseline latency (~4s) aur error rate record karna.

Days 22 – 30: Surget.AI Hybrid Agent

SOTA Result

- DOM/UIA Grounding + Workflow Memory DAG add karna.
- Replay speedup (<10ms) aur zero-token replay prove karna.
- Comparison table & latency curves generate karna.
- *Task:* Final Research Paper draft ready karna!

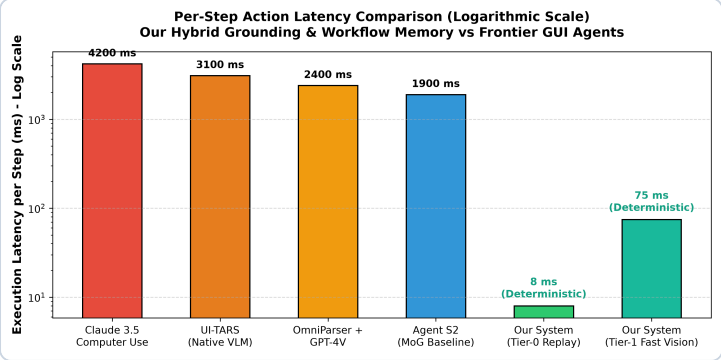


Figure: Goal of this roadmap — Bringing step latency down from 4,200ms to <10ms on verified repeated workflows.

© 2026 Surget.AI Autonomous Workforce Intelligence. All rights reserved.